

컴퓨터 비전

동양미래대학교
인공지능소프트웨어학과
백찬은 교수

Vision Transformer

01

트랜스포머와
비전 트랜스포머

02

비전 트랜스포머 - 분류

03

기존 모델을 활용한
비전 트랜스포머의 분류, 검출, 분할





트랜스포머와 비전 트랜스포머

Transformer

- 논문 'Attention Is All You Need'(2017)에서 소개된 딥러닝 모델
- 순차적 처리 없이도 입력 시퀀스의 관계를 효과적으로 학습할 수 있도록 설계되었음

주요 특징

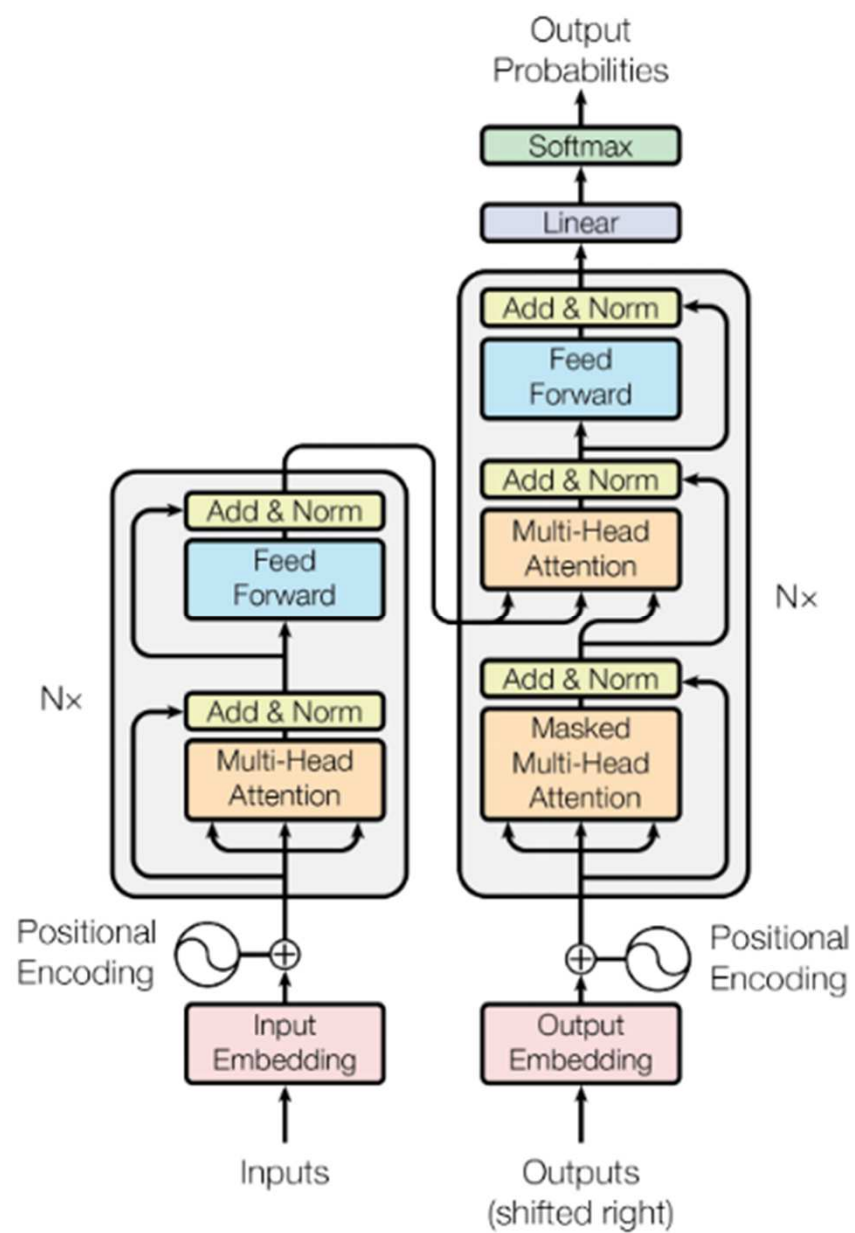
- **Self-Attention Mechanism:** 입력 시퀀스 내의 모든 단어 간의 관계를 동적으로 학습함
- **Encoder-Decoder 구조 사용**
- **병렬 처리:** 전체 시퀀스를 동시에 처리할 수 있어 RNN(순차적으로 처리해야 함)보다 학습속도가 빠름
- **다양한 활용 분야:** 자연어 처리, 컴퓨터 비전(ViT), 음성 처리 등 다양한 분야에 활용됨

Transformer의 구조

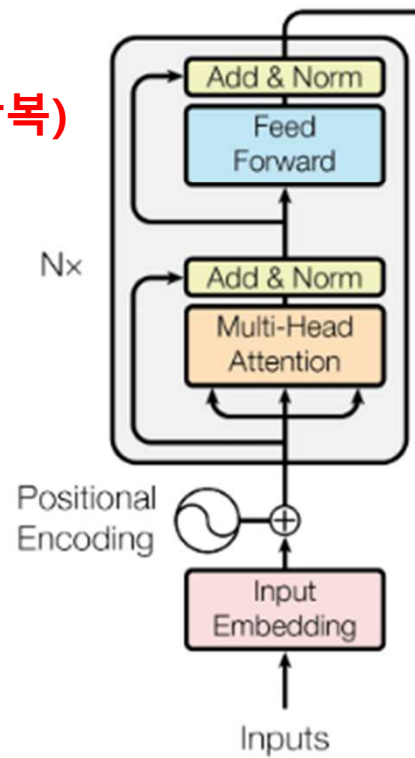
(입력 데이터) -> 토큰나이저 -> 임베딩 레이어 -> 인코더 N개 -> 디코더 N개 -> 출력층 -> (출력 결과)

*디코더는 순차적인 생성 작업을 할 때 필요하며, 단순 분류 목적(예: BERT)일 경우에는 필요하지 않음.

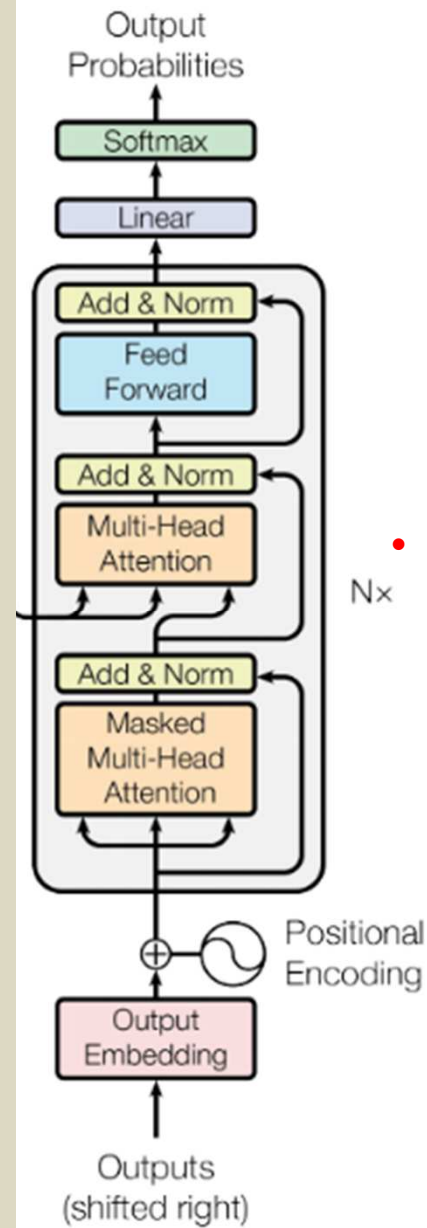
- **토큰나이저(Tokenizer):** 입력 텍스트를 토큰화하고 정수 ID 시퀀스로 인코딩함
- **임베딩 레이어(Embedding layer):** 정수 ID 시퀀스를 각 ID에 해당하는 고차원 벡터(임베딩)로 변환하고, 위치 정보를 더함
- **Encoder:** 전체 시퀀스를 받아 각 단어가 문장 내 다른 모든 단어들과 어떤 관계를 맺고 있는지 학습함. 단어의 문맥적 의미를 보다 풍부하게 파악할 수 있음. 여러 인코더 층이 쌓여 있음
- **Decoder:** 이전에 생성한 단어들과 인코더의 출력을 입력받아 다음 단어를 순차적으로 예측함. 여러 디코더 층이 쌓여 있음
- **출력층(Output Layer):** 문제 유형에 맞는 최종 결과를 생성함



• 왼쪽 - 인코더 스택(n회 반복)



• 오른쪽 - 디코더 스택(n회 반복)



Multi-Head Self-Attention

Self-Attention

입력 시퀀스 내의 **모든 단어들끼리의 관계**(의존성)를 동적으로 파악하는 매커니즘

단어마다 문맥을 반영한 표현을 만듦

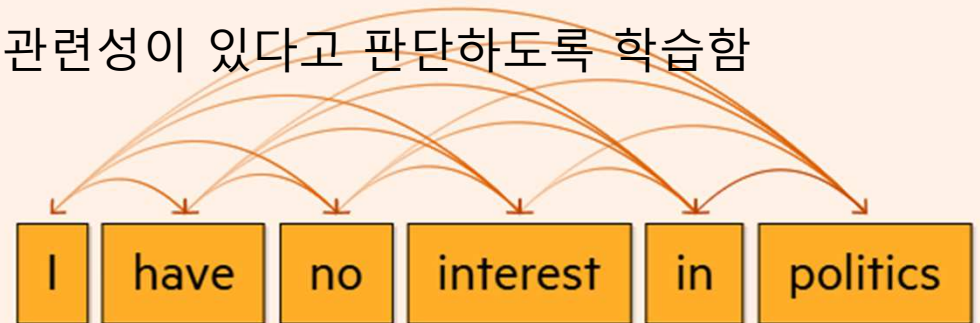
!!전체 토큰간의 관계를 동시에 계산함(병렬 계산)

"이 단어가 문장 내에 있는 다른 어떤 단어들과 얼마나 관련이 있을까?"를 계산하는 것

"The animal didn't cross the street because it was too tired."

단어 'it'이 가리키는 것이 무엇일까?

-> Self-Attention은 it이 street보다는 animal과 더 높은 관련성이 있다고 판단하도록 학습함



Multi-Head Self-Attention

Multi-Head Self-Attention

Self-Attention을 다양한 방식으로 여러번 실행해서 정보의 시야를 넓힘(다양한 시각으로 문장을 바라봄)

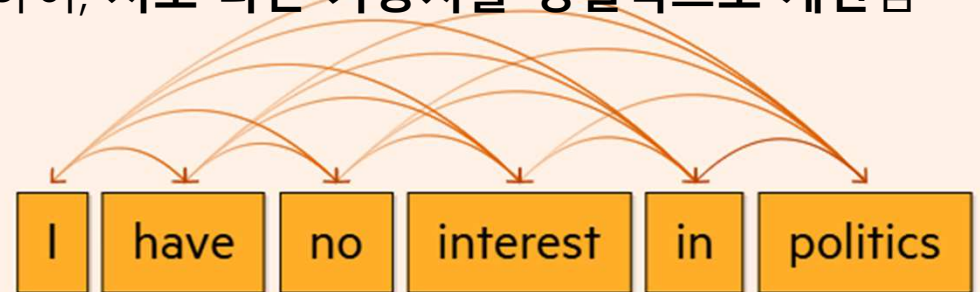
(예) head1 - 문법적 관계

(예) head2 - 의미적 유사성

(예) head3 - 위치 정보

...

Multi-Head Self-Attention은 **self-attention**을 여러번하여, 서로 다른 가중치를 병렬적으로 계산함



다양한 transformer 기반 모델

- **BERT**(Bidirectional Encoder Representations from Transformers)
 - 트랜스포머 기반 양방향 인코더 표현 모델
- **GPT**(Generative Pre-trained Transformer)
 - 생성형 사전학습 트랜스포머 모델
- **T5**(Text-to-Text Transfer Transformer)
 - 텍스트-투-텍스트 전이 학습 트랜스포머 모델
- **BART**(Bidirectional and Auto-Regressive Transformer)
 - 양방향 자기회귀 트랜스포머 모델

비전 트랜스포머

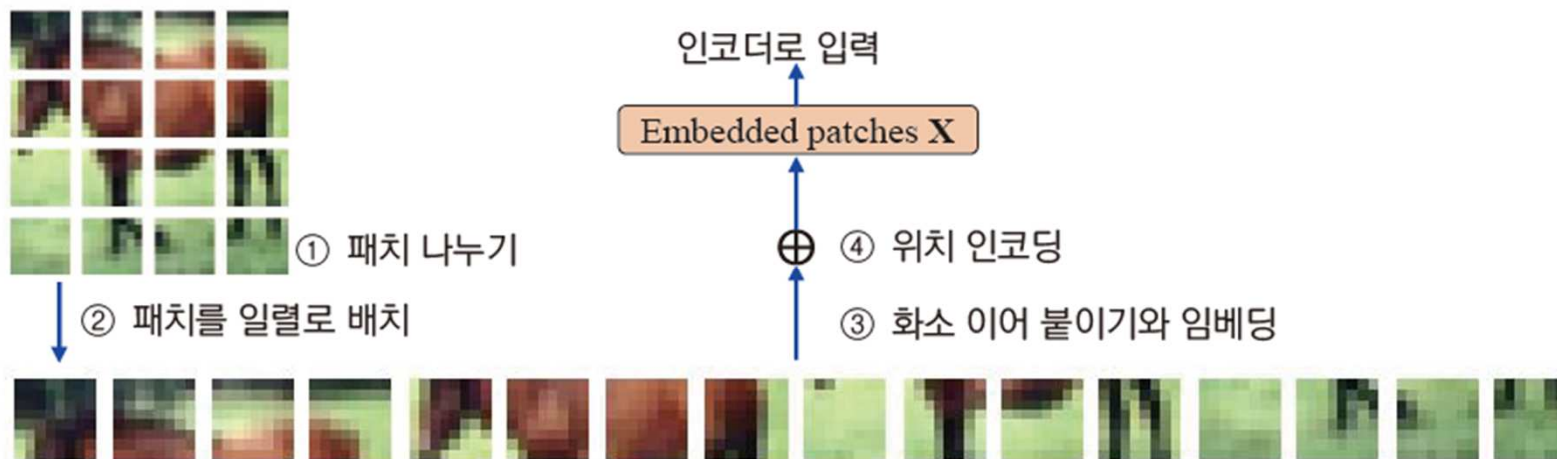
- **ViT(Vision Transformer)**

- 트랜스포머 구조를 영상 데이터에 알맞게 변경한 모델
- 문장은 단어가 순서대로 배열된 1차원 구조인데, 영상은 화소가 2차원 구조로 배열
- 단어는 이산적인데 영상은 연속적
- 문장을 구성하는 단어는 수십 개인데 영상을 구성하는 화소는 수만~수백만 개
- 단어는 분할되어 있지만 영상은 물체와 배경이 복잡하게 섞여 있음
- 물체는 크기(스케일), 회전, 이동에 심한 변화가 발생

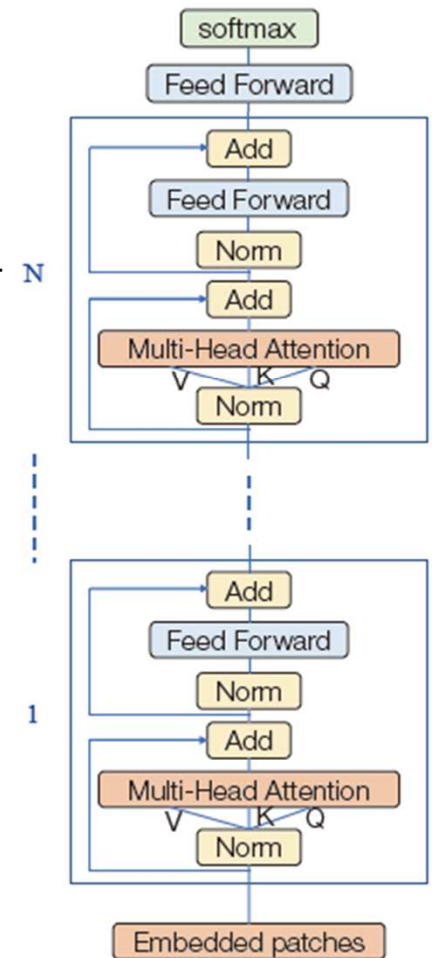
비전 트랜스포머

• ViT(Vision Transformer)의 구조

- 영상 분류에서는 Class의 확률 벡터를 출력하면 되기 때문에 인코더만 사용
- 인코더는 특징 맵을 softmax를 통해 부류 확률 벡터로 변환하여 출력



(a) 인코더 입력 텐서 준비 과정



(b) 신경망 구조

그림 11-18 영상 분류를 위한 ViT 비전 트랜스포머



비전 트랜스포머 - 분류

(실습) 비전 트랜스포머 - 분류

프로그램 11-1

CIFAR-10을 분류하는 비전 트랜스포머 구현하기

```
01 import tensorflow as tf
02 from tensorflow import keras
03 from tensorflow.keras import layers
04 from tensorflow.keras.losses import SparseCategoricalCrossentropy
05 from tensorflow.keras.optimizers import Adam
06
07 (x_train,y_train),(x_test,y_test)=keras.datasets.cifar10.load_data()
08
09 n_class=10 # 분류 수
10 img_siz=(32,32,3) # 영상의 크기
11
12 patch_siz=4 # 패치 크기
13 p2=(img_siz[0]//patch_siz)**2 # 패치 개수
14 d_model=64 # 임베딩 벡터 차원
15 h=8 # 헤드 개수
16 N=6 # 인코더 블록의 개수
```

```

17
18 class Patches(layers.Layer):
19     def __init__(self, patch_size):
20         super(Patches, self).__init__()
21         self.p_siz=patch_size
22
23     def call(self, img):
24         batch_size=tf.shape(img)[0]
25         patches=tf.image.extract_patches(images=img,sizes=[1,self.p_siz,self.p_
26             siz,1],strides=[1,self.p_siz,self.p_siz,1],rates=[1,1,1,1],padding="
27             VALID")
28         patch_dims=patches.shape[-1]
29         patches=tf.reshape(patches,[batch_size,-1,patch_dims])
30         return patches
31
32 class PatchEncoder(layers.Layer):
33     def __init__(self,p2,d_model):
34         # p2: 패치 개수
35         super(PatchEncoder,self).__init__()
36         self.p2=p2
37         self.projection=layers.Dense(units=d_model)
38         self.position_embedding=layers.Embedding(input_dim=p2,output_dim=d_
39             model)
40
41     def call(self,patch):
42         positions=tf.range(start=0,limit=self.p2,delta=1)
43         encoded=self.projection(patch)+self.position_embedding(positions)
44         return encoded

```

```

41
42 def create_vit_classifier():
43     input=layers.Input(shape=(img_siz))
44     nor=layers.Normalization()(input)
45
46     patches=Patches(patch_siz)(nor)           # 패치 생성
47     x=PatchEncoder(p2,d_model)(patches)       # 패치 인코딩
48
49     for _ in range(N):                       # 다중 인코더 블록
50         x1=layers.LayerNormalization(epsilon=1e-6)(x)   # 층 정규화
51         x2=layers.MultiHeadAttention(num_heads=h,key_dim=d_model//h,dropout
           =0.1)(x1,x1)                               # MHA
52         x3=layers.Add()(x2,x1)                   # 지름길 연결
53         x4=layers.LayerNormalization(epsilon=1e-6)(x3)   # 층 정규화
54         x5=layers.Dense(d_model*2,activation=tf.nn.gelu)(x4)
55         x6=layers.Dropout(0.1)(x5)
56         x7=layers.Dense(d_model,activation=tf.nn.gelu)(x6)
57         x8=layers.Dropout(0.1)(x7)
58         x=layers.Add()(x8,x3)                   # 지름길 연결
59
60     x=layers.LayerNormalization(epsilon=1e-6)(x)
61     x=layers.Flatten()(x)
62     x=layers.Dropout(0.5)(x)
63     x=layers.Dense(2048,activation=tf.nn.gelu)(x)
64     x=layers.Dropout(0.5)(x)
65     x=layers.Dense(1024,activation=tf.nn.gelu)(x)
66     x=layers.Dropout(0.5)(x)
67     output=layers.Dense(n_class,activation='softmax')(x)
68
69     model=keras.Model(inputs=input,outputs=output)
70     return model

```



```

71
72 model=create_vit_classifier()
73 model.layers[1].adapt(x_train)
74
75 model.compile(optimizer=Adam(),loss=SparseCategoricalCrossentropy(from_logits
=True),metrics=['accuracy'])
76 hist=model.fit(x_train,y_train,batch_size=128,epochs=100,validation_data=(x_
test,y_test),verbose=1)
77
78 res=model.evaluate(x_test,y_test,verbose=0)
79 print('정확률=',res[1]*100)
80
81 import matplotlib.pyplot as plt
82
83 plt.plot(hist.history['accuracy'])
84 plt.plot(hist.history['val_accuracy'])
85 plt.title('Accuracy graph')
86 plt.ylabel('Accuracy')
87 plt.xlabel('Epoch')
88 plt.legend(['Train','Validation'])
89 plt.grid()
90 plt.show()
91
92 plt.plot(hist.history['loss'])
93 plt.plot(hist.history['val_loss'])
94 plt.title('Loss graph')
95 plt.ylabel('Loss')
96 plt.xlabel('Epoch')
97 plt.legend(['Train','Validation'])
98 plt.grid()
99 plt.show()

```

시간관계상 epochs 값을 5로 변경하여 실습 진행하기!

Epoch 1/100

391/391 - 30s - loss: 1.8080 - accuracy: 0.3693 - val_loss: 1.3451 - val_accuracy: 0.5064 - 30s/epoch - 77ms/step

Epoch 2/100

391/391 - 25s - loss: 1.4014 - accuracy: 0.4966 - val_loss: 1.1915 - val_accuracy: 0.5708 - 25s/epoch - 64ms/step

Epoch 3/100

391/391 - 26s - loss: 1.2445 - accuracy: 0.5565 - val_loss: 1.1240 - val_accuracy: 0.5914 - 26s/epoch - 67ms/step

...

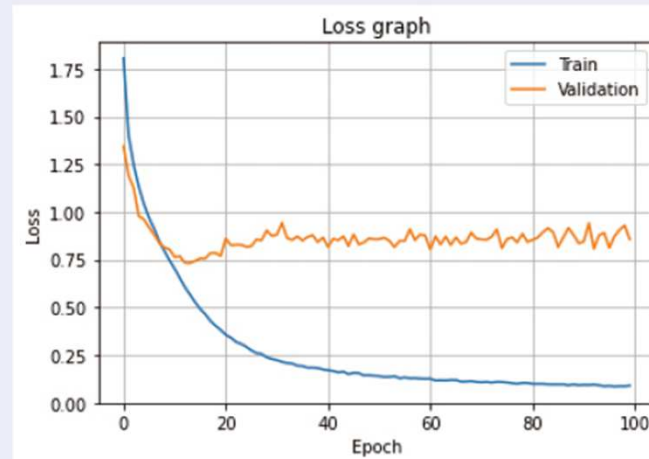
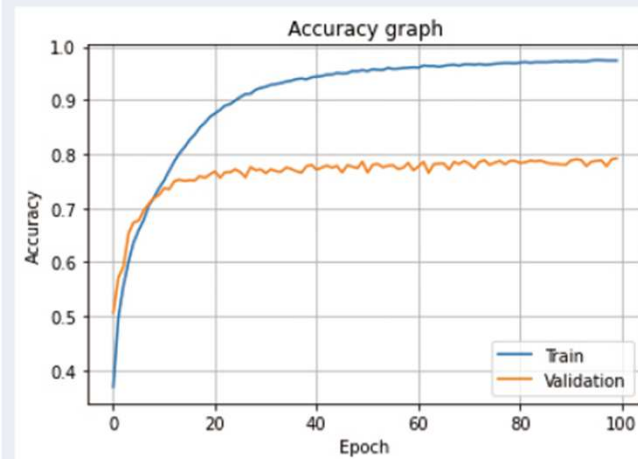
Epoch 99/100

391/391 - 25s - loss: 0.0864 - accuracy: 0.9726 - val_loss: 0.9296 - val_accuracy: 0.7903 - 25s/epoch - 64ms/step

Epoch 100/100

391/391 - 25s - loss: 0.0896 - accuracy: 0.9725 - val_loss: 0.8569 - val_accuracy: 0.7918 - 25s/epoch - 64ms/step

정확률= 79.18000221252441



(실습) 비전 트랜스포머 - 분류, 영상 확대 및 데이터 증강으로 성능 향상

32*32 영상을 72*72로 확대하고 데이터 증강을 추가하여 성능 향상 시도

프로그램 11-2

CIFAR-10을 분류하는 비전 트랜스포머: 영상 확대와 데이터 증강으로 성능 향상하기

01~08행은 [프로그램 11-1]과 같음

```
09  n_class=10                      # 분류 수
10  img_siz=(32,32,3)
11  img_expanded_siz=(72,72,3)
12
13  patch_siz=6                      # 패치 크기
14  p2=(img_expanded_siz[0]//patch_siz)**2  # 패치 개수
15  d_model=64                      # 임베딩 벡터 차원
16  h=8                             # 헤드 개수
17  N=6                             # 인코더 블록의 개수
```

18~42행은 [프로그램 11-1]의 17~41행과 같음

```

43 def create_vit_classifier():
44     input=layers.Input(shape=(img_siz))
45     nor=layers.Normalization()(input)
46     exp=layers.Resizing(img_expanded_siz[0],img_expanded_siz[1])(nor)
47
48     x=layers.RandomFlip('horizontal')(exp)
49     x=layers.RandomRotation(factor=0.02)(x)
50     x=layers.RandomZoom(height_factor=0.2, width_factor=0.2)(x)
51
52     patches=Patches(patch_siz)(x)           # 패치 생성
53     x=PatchEncoder(p2,d_model)(patches)     # 패치 인코딩

```

이후는 [프로그램 11-1]의 48~99행과 같음

Epoch 1/100

391/391 - 63s - loss: 1.9473 - accuracy: 0.3414 - val_loss: 1.3933 - val_accuracy: 0.4899 - 63s/epoch - 161ms/step

Epoch 2/100

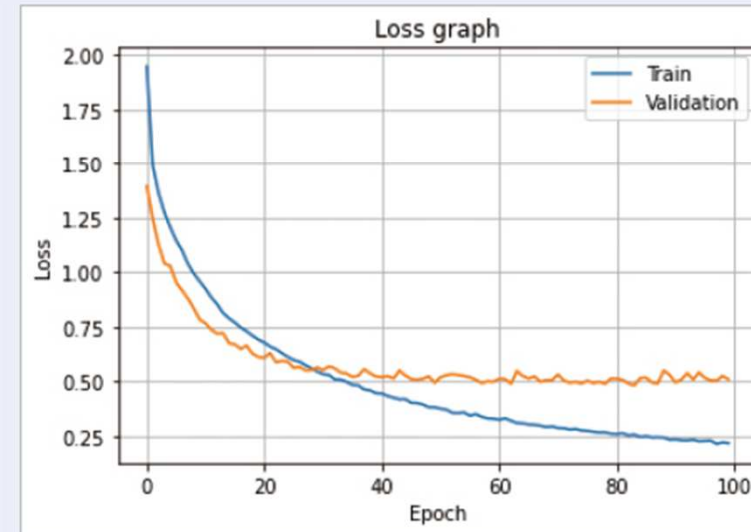
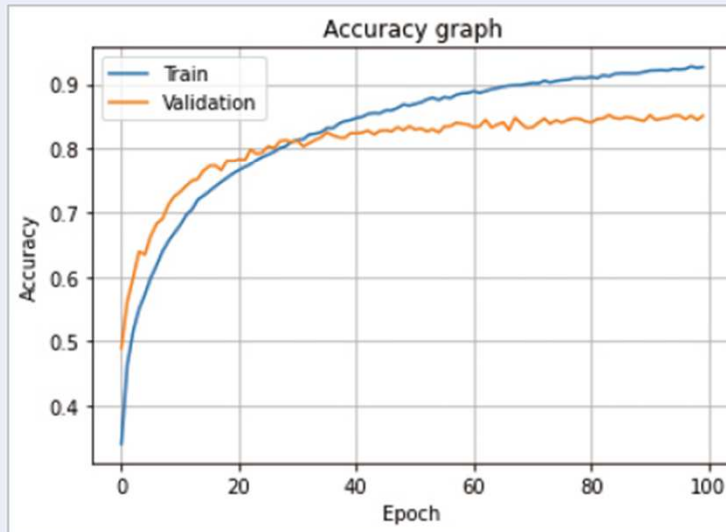
391/391 - 57s - loss: 1.4903 - accuracy: 0.4645 - val_loss: 1.2458 - val_accuracy: 0.5622 - 57s/epoch - 147ms/step

...

Epoch 100/100

391/391 - 58s - loss: 0.2170 - accuracy: 0.9268 - val_loss: 0.5090 - val_accuracy: 0.8511 - 58s/epoch - 147ms/step

정확률= 85.11000275611877





기존 모델을 활용한
비전 트랜스포머의 분류, 검출, 분할

(실습) 허깅페이스의 ViT 활용

```
1  from transformers import ViTImageProcessor, ViTForImageClassification
2  from PIL import Image
3  import torch
4  import matplotlib.pyplot as plt
5
6  # 1. 이미지 불러오기
7  imgs = [
8      Image.open('BSDS_242078.jpg'),
9      Image.open('BSDS_361010.jpg'),
10     Image.open('BSDS_376001.jpg')
11 ]
12
13 # 2. ViT feature extractor + model
14 processor = ViTImageProcessor.from_pretrained("google/vit-base-patch16-224")
15 model = ViTForImageClassification.from_pretrained("google/vit-base-patch16-224")
16
17 # 3. 이미지 전처리
18 inputs = processor(images=imgs, return_tensors="pt")
19
```


(실습) 허깅페이스의 ViT 활용

```
20 # 4. 모델 추론
21 with torch.no_grad():
22     outputs = model(**inputs)
23     logits = outputs.logits
24     probs = logits.softmax(dim=-1)
25
26 # 5. 결과 출력
27 for i in range(len(imgs)):
28     plt.imshow(imgs[i]); plt.axis("off"); plt.show()
29
30     pred_idx = int(torch.argmax(probs[i]))
31     pred_label = model.config.id2label[pred_idx]
32     pred_prob = float(probs[i][pred_idx] * 100)
33
34     print(f"{i}번째 이미지 예측:")
35     print(f"    ▶ 클래스: {pred_label}")
36     print(f"    ▶ 확률: {pred_prob:.2f}%\n")
```


(실습) 허깅페이스의 ViT 활용



0번째 이미지 예측:
▶ 클래스: umbrella
▶ 확률: 98.76%



1번째 이미지 예측:
▶ 클래스: croquet ball
▶ 확률: 3.90%

(실습) 허깅페이스의 ViT 활용



2번째 이미지 예측:

- ▶ 클래스: hay
- ▶ 확률: 18.51%

비전 트랜스포머 - 검출과 분할

DETR(Detection with Transformer)

- : 트랜스포머의 원래 구조를 유지하면서 박스 집합을 예측할 수 있게 개조된 모델
- 허깅페이스에서 제공하는 모델을 가져와 사용할 수 있음

(실습) 허깅페이스의 DETR 모델 활용

```
1 #python 3.9 버전 사용
2 #가상환경을 파이썬 3.9로 만들어서 사용하기
3 !pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118
4 !pip install transformers timm opencv-python numpy==1.26.4
5
6 from transformers import DetrImageProcessor, DetrForObjectDetection
7 from PIL import Image
8 import torch
9 import numpy as np
10 import cv2 as cv
11
12 img = Image.open("BSDS_361010.jpg")
13
14 processor = DetrImageProcessor.from_pretrained("facebook/detr-resnet-50")
15 model = DetrForObjectDetection.from_pretrained("facebook/detr-resnet-50")
16
17 inputs = processor(images=img, return_tensors="pt")
18 outputs = model(**inputs)
19
```


(실습) 허깅페이스의 DETR 모델 활용

```
20 target_sizes = torch.tensor([img.size[:-1]])
21 results = processor.post_process_object_detection(outputs, threshold=0.7, target_sizes=target_sizes)[0]
22
23 im = cv.cvtColor(np.array(img), cv.COLOR_RGB2BGR)
24 colors = np.random.uniform(0, 255, size=(100, 3))
25
26 for score, label, box in zip(results["scores"], results["labels"], results["boxes"]):
27     x_min, y_min, x_max, y_max = map(int, box.tolist())
28     color = colors[label.item() % 100]
29     name = model.config.id2label[label.item()]
30     cv.rectangle(im, (x_min, y_min), (x_max, y_max), color, 2)
31     cv.putText(im, f"{name} {score.item():.2f}", (x_min, y_min - 5),
32                cv.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)
33
34 cv.imshow("DETR", im)
35 cv.waitKey(0)
36 cv.destroyAllWindows()
```

(실습) 허깅페이스의 DETR 모델 활용



(실습) 허깅페이스의 CLIP 모델 활용

```
1  from transformers import CLIPProcessor, CLIPModel
2  from PIL import Image
3
4  img=Image.open('BSDS_361010.jpg')
5
6  processor=CLIPProcessor.from_pretrained('openai/clip-vit-base-patch32')
7  model=CLIPModel.from_pretrained('openai/clip-vit-base-patch32')
8
9  captions=['Two horses are running on grass',
10           'Students are eating',
11           'Croquet playing on horses',
12           'Golf playing on horses']
13  inputs=processor(text=captions,images=img,return_tensors='pt',padding=True)
14  res=model(**inputs)
15
16  import matplotlib.pyplot as plt
17  plt.imshow(img); plt.xticks([]); plt.yticks([]); plt.show()
18
19  logits=res.logits_per_image
20  probs=logits.softmax(dim=1)
21  for i in range(len(captions)):
22      print(captions[i],': ', '{:.2f}'.format(float(probs[0,i]*100.0)))
```

(실습) 허깅페이스의 CLIP 모델 활용



```
Two horses are running on grass : 10.12  
Students are eating : 0.00  
Croquet playing on horses : 51.64  
Golf playing on horses : 38.24
```

```
In [71]:
```